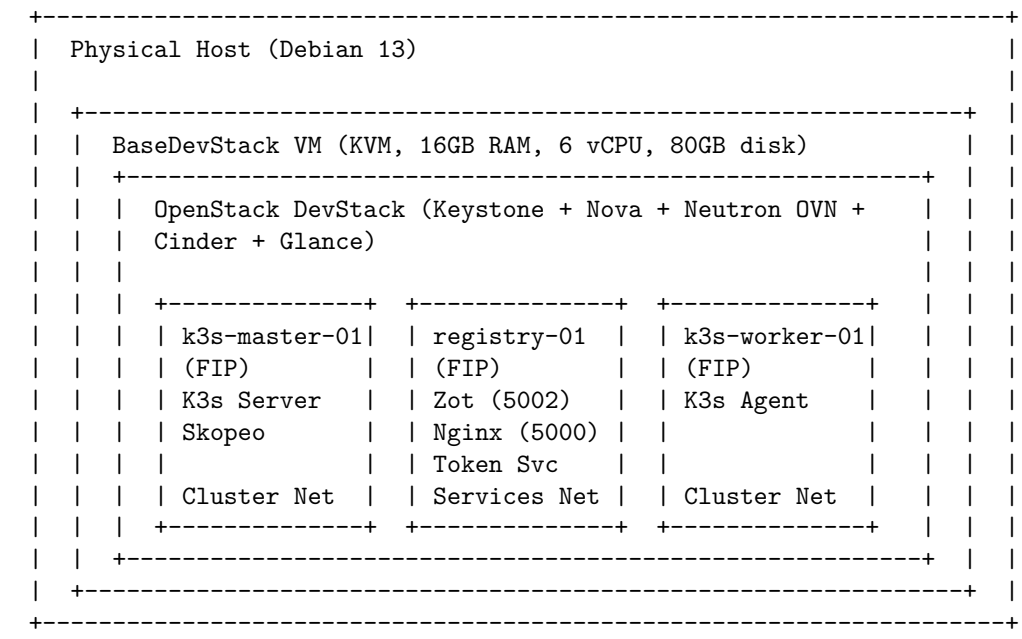
Cloud Engineering Final Project — Complete Workflow Report

Overview

This project deploys a **fully air-gapped Kubernetes (K3s) cluster on top of OpenStack**, with an **authenticated private OCI registry (Zot)** and **Keystone-backed token authentication** — all running inside nested VMs on a single physical host. The architecture demonstrates production-relevant patterns: OpenStack IaaS provisioning, private registry with identity federation, and zero-internet-dependency container orchestration.

Architecture at a Glance



Network topology: - **Cluster Network** (`10.10.0.0/24`) — K3s master + workers - **Services Network** (`10.20.0.0/24`) — Registry + Auth Proxy - **Public/External** (`192.168.122.200/29`) — Floating IPs via br-ex - **Host access** — Route via `192.168.122.0/24` through DevStack VM

Step-by-Step Workflow

Step 00 — Prerequisites & Configuration

Files: `config.env`, `lib/common.sh`, `source_env.sh`

Before running any script, the host must have: - **Libvirt/KVM** installed and running - **Ubuntu 24.04 cloud image** (`noble-server-cloudimg-amd64.img`) in the project directory - **SSH keypair** at `~/.ssh/id_ed25519` - **Pre-downloaded binaries**: `k3s`, `k3s-airgap-images-amd64.tar`, `zot-linux-amd64`, `skopeo`

`config.env` defines: | Variable | Value | Purpose | |—|—|—| | `BASE_IMAGE` | `./noble-server-cloudimg-amd64.img` | Ubuntu 24.04 qcow2 | | `VM_RAM` | 16284 MB | DevStack VM memory | | `VM_VCPUS` | 6 | DevStack VM CPUs | | `DISK_SIZE` | 80G | DevStack VM disk | | `K3S_BINARY` | `./k3s` | Pre-downloaded K3s binary | | `K3S_AIRGAP_IMAGES` | `./k3s-airgap-images-amd64.tar` | Pre-pulled images tar | | `WORKER_COUNT` | 1 | Number of K3s worker nodes |

`lib/common.sh` provides shared functions used by all scripts (03-11): - `load_env` — sources `devstack_env` and sets defaults for all project variables - `persist_env` — writes key=value to `devstack_env` (state file across scripts) - `vm_ssh` / `vm_openstack` — execute commands on DevStack VM as admin or tenant - `wait_for_ssh` / `wait_for_server_status` / `wait_for_fixed_ip_for_net` — polling helpers - `require_env` / `require_file` — pre-flight validation - `log` / `log_section` / `fail` — structured logging

---

Step 01 — Create the Base DevStack VM

File: `01_create_vm.sh`

Creates the foundational KVM virtual machine that will host OpenStack DevStack.

Key commands:

```
# Destroy existing VM if present
sudo virsh destroy BaseDevStack
sudo virsh undefine BaseDevStack --remove-all-storage

# Create qcow2 disk backed by Ubuntu cloud image
sudo qemu-img create -f qcow2 -F qcow2 -b noble-server-cloudimg-amd64.img \
    /var/lib/libvirt/images/BaseDevStack.qcow2 80G

# Generate cloud-init ISO (SSH key, password, packages)
genisoimage -output cidata.iso -volid cidata -joliet -rock user-data meta-data

# Deploy VM with TWO network interfaces (critical for Neutron)
sudo virt-install \
```

```
    --name BaseDevStack --memory 16284 --vcpus 6 \
    --cpu host-passthrough --os-variant ubuntu24.04 \
    --disk path=BaseDevStack.qcow2,format=qcow2 \
    --disk path=cidata.iso,device=cdrom \
    --import \
    --network network=default,model=virtio \    # enp1s0: management/SSH
    --network network=default,model=virtio \    # enp2s0: Neutron provider bridge
    --graphics none --noautoconsole

# Poll for IP and persist to devstack_env
IP_ADDR=$(sudo virsh domifaddr BaseDevStack | grep -oE "\b([0-9]{1,3}\.){3}[0-9]{1,3}\b" | h
echo "export VM_IP=$IP_ADDR" > ./devstack_env
```

Why two NICs? The first (`enp1s0`) is the NAT management interface for SSH. The second (`enp2s0`) is left unconfigured so DevStack can bind it as the OVN external bridge (`br-ex`) for floating IPs.

Output: `devstack_env` file with `VM_IP` persisted.

Step 02 — Initialize the DevStack VM

File: `02_init_vm.sh`

Prepares the VM for DevStack installation. Must be run after Step 01.

Key commands:

```
source ./devstack_env   # Load VM_IP

# Fix DNS (systemd-resolved conflicts with DevStack)
sudo systemctl disable --now systemd-resolved
sudo rm -f /etc/resolv.conf
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf

# Prepare second interface for Neutron provider network
sudo ip link set enp2s0 up
sudo ip addr flush dev enp2s0

# Install prerequisites and clone DevStack
sudo apt update && sudo apt install -y git
git clone https://opendev.org/openstack/devstack ~/devstack

# Transfer local.conf (DevStack configuration)
scp local.conf ubuntu@$VM_IP:/home/ubuntu/devstack/local.conf

# Fix HOST_IP in local.conf to match actual management IP
```

```
CURRENT_IP=$(hostname -I | awk '{print $1}')
sed -i "s/^HOST_IP=.*/HOST_IP=$CURRENT_IP/" ~/devstack/local.conf
```

`local.conf` highlights: - Passwords: `nomoresecret` (all services) - Neutron: OVN with Geneve tunneling, `enp2s0` as public interface - Floating range: `192.168.122.200/29` - OVS bridge mapping: `public:br-ex` - Cinder: minimal (20G backing file) - Libvirt type: KVM (nested virtualization)

Manual step: After this script, the user must SSH in and run `./stack.sh` manually. Once complete, a **Libvirt snapshot** named `PostStack_Ready` should be taken for fast reverts.

Step 03 — Bootstrap OpenStack Resources

File: `03_openstack_bootstrap.sh`

Creates the OpenStack project, user, networks, and application credentials needed by the K3s cluster.

Key commands (executed inside DevStack VM via `vm_openstack`):

```
# Upload Ubuntu image to Glance
openstack image create 'Ubuntu24' --file ~/ubuntu-noble.img \
    --disk-format qcow2 --container-format bare --public

# Create project + user
openstack project create --description "K3s Deployment" k3s-project
openstack user create --project k3s-project --password usersecret k3s-user
openstack role add --project k3s-project --user k3s-user member

# Set quotas
openstack quota set --ram 32768 --cores 20 --instances 20 --floating-ips 20 k3s-project

# Create router with external gateway
openstack router create k3s-router
openstack router set --external-gateway public k3s-router

# Create Cluster Network (10.10.0.0/24)
openstack network create --project k3s-project k3s-cluster-net
openstack subnet create --project k3s-project --network k3s-cluster-net \
    --subnet-range 10.10.0.0/24 --gateway 10.10.0.1 --dns-nameserver 1.1.1.1 k3s-cluster-sub

# Create Services Network (10.20.0.0/24)
openstack network create --project k3s-project k3s-services-net
openstack subnet create --project k3s-project --network k3s-services-net \
    --subnet-range 10.20.0.0/24 --gateway 10.20.0.1 --dns-nameserver 1.1.1.1 k3s-services-su
```

```
# Attach subnets to router
openstack router add subnet k3s-router k3s-cluster-subnet
openstack router add subnet k3s-router k3s-services-subnet

# Create Application Credential for Auth Proxy
openstack application credential create auth-proxy-system --unrestricted -f json
```

Also generates: - `k3s-user-openrc.sh` — OpenRC for the k3s-user tenant (copied to VM and host) - `AUTH_PROXY_APP_CRED_ID` / `AUTH_PROXY_APP_CRED_SECRET` — persisted to `devstack_env`

Output: OpenStack project with two tenant networks, a router, and an application credential for the auth proxy.

Step 04 — Create the K3s Master VM

File: `04_k3s_master_vm.sh`

Provisions the K3s master node as an OpenStack instance on the cluster network.

Key commands:

```
# Upload SSH public key as OpenStack keypair
openstack keypair create --public-key ~/host_instance_key.pub k3s-key

# Create security group for master
openstack security group create k3s-master-sg
# Rules: SSH (22) from host, K8s API (6443) from host, all traffic from cluster CIDR
openstack security group rule create --remote-ip 192.168.122.0/24 --protocol tcp --dst-port
openstack security group rule create --remote-ip 192.168.122.0/24 --protocol tcp --dst-port
openstack security group rule create --remote-ip 10.10.0.0/24 --protocol tcp k3s-master-sg
openstack security group rule create --remote-ip 10.10.0.0/24 --protocol udp k3s-master-sg
openstack security group rule create --remote-ip 10.10.0.0/24 --protocol icmp k3s-master-sg

# Create master server
openstack server create --flavor m1.medium --image Ubuntu24 \
    --network k3s-cluster-net --security-group k3s-master-sg \
    --key-name k3s-key k3s-master-01

# Allocate and assign floating IP
openstack floating ip create public
openstack server add floating ip k3s-master-01 <FIP>

# Critical: configure host routing to floating range through DevStack VM
sudo ip route replace 192.168.122.200/29 via $VM_IP
```

```
# Configure br-ex on DevStack VM (enable floating IP forwarding)
# Runs configure_br_ex.sh inside the VM with admin OpenRC
```

configure_br_ex.sh does:

```
# Discover public subnet gateway IP
PUBLIC_GW_IP=$(openstack subnet show public-subnet -f value -c gateway_ip)
PUBLIC_CIDR=$(openstack subnet show public-subnet -f value -c cidr)

# Configure the OVS bridge
sudo ip link set br-ex up
sudo ip addr replace $PUBLIC_GW_IP/$PREFIX dev br-ex
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w net.ipv4.conf.all.rp_filter=0    # Allow asymmetric routing
```

Output: Master VM with floating IP, persisted as `MASTER_FIP`, `MASTER_INTERNAL_IP`.

Step 05 — Create the Registry VM

File: `05_registry_vm.sh`

Provisions the registry/auth-proxy VM on the services network with a Cinder volume for persistent storage.

Key commands:

```
# Create security group for registry
openstack security group create registry-sg
# Rules: SSH + port 5000 from cluster/services CIDR, port 5000 from 0.0.0.0/0 (external)
openstack security group rule create --remote-ip 0.0.0.0/0 --protocol tcp --dst-port 5000 re

# Create 20GB Cinder volume
openstack volume create --size 20 registry-01-volume

# Create server on services network
openstack server create --flavor m1.small --image Ubuntu24 \
    --network k3s-services-net --security-group registry-sg \
    --key-name k3s-key registry-01

# Attach volume
openstack server add volume registry-01 registry-01-volume

# Allocate floating IP (needed for Auth Proxy external access)
openstack floating ip create public
openstack server add floating ip registry-01 <FIP>
```

Output: Registry VM with floating IP, persisted as `REGISTRY_FIP`, `REGISTRY_INTERNAL_IP`.

Step 06 — Install Zot Registry + Import Airgap Images

File: `06_zot_install.sh`

Installs the Zot OCI distribution registry on the Registry VM and populates it with K3s airgap images.

Key commands:

```
# SCP Zot binary to Registry VM
scp zot-linux-amd64 ubuntu@$REGISTRY_FIP:~/zot

# Inside Registry VM:
# Create zot system user
sudo adduser --system --group --no-create-home --disabled-password zot

# Format and mount Cinder volume
sudo mkfs.ext4 -F /dev/vdb
sudo mkdir -p /mnt/registry
echo "UUID=... /mnt/registry ext4 defaults,nofail 0 2" >> /etc/fstab
sudo mount /mnt/registry
sudo chown zot:zot /mnt/registry

# Install Zot binary
sudo install -m 0755 ~/zot /usr/local/bin/zot

# Create Zot config (listening on 0.0.0.0:5000, no auth yet)
cat <<JSON | sudo tee /etc/zot/config.json
{
  "distSpecVersion": "1.1.0",
  "storage": { "rootDirectory": "/mnt/registry/data" },
  "http": { "address": "0.0.0.0", "port": "5000" },
  "log": { "level": "info", "output": "/var/log/zot/zot.log" },
  "extensions": { "search": { "enable": true }, "ui": { "enable": true } }
}
JSON

# Create systemd service
sudo tee /etc/systemd/system/zot.service
# ExecStart=/usr/local/bin/zot serve /etc/zot/config.json
sudo systemctl enable --now zot

# Install Skopeo on Master VM
scp skopeo ubuntu@$MASTER_FIP:~/skopeo
sudo install -m 0755 ~/skopeo /usr/local/bin/skopeo
```

```
# Import airgap images from Master VM to Zot
scp k3s-airgap-images-amd64.tar ubuntu@$MASTER_FIP:/home/ubuntu/k3s-airgap-images.tar
scp import_images.sh ubuntu@$MASTER_FIP:~/import_images.sh
ssh ubuntu@$MASTER_FIP "REGISTRY_INTERNAL_IP=$REGISTRY_INTERNAL_IP bash ~/import_images.sh"
```

import_images.sh logic:

```
# Extract image list from tar manifest
IMAGES=$(tar -xOf k3s-airgap-images.tar manifest.json | jq -r '.[].RepoTags[]')
# Push each image to Zot via Skopeo
for IMG in $IMAGES; do
    skopeo copy --dest-tls-verify=false \
        docker-archive:k3s-airgap-images.tar:docker.io/$IMG \
        docker://$REGISTRY_INTERNAL_IP:5000/${IMG#*/}
done
```

Output: Zot running on `0.0.0.0:5000` with all K3s system images pre-loaded.

Step 07 — Auth Proxy VM Alias

File: `07_auth_proxy_vm.sh`

This is a **no-op script** — the Auth Proxy was originally planned as a separate VM, but was merged into the Registry VM to reduce resource usage. It simply aliases the registry IPs:

```
persist_env AUTH_PROXY_FIP "$REGISTRY_FIP"
persist_env AUTH_PROXY_INTERNAL_IP "$REGISTRY_INTERNAL_IP"
```

This ensures downstream scripts (08, 10, 11) work without modification.

Step 08 — Install Auth Proxy (Nginx + Token Service)

File: `08_auth_proxy_install.sh`

Installs the authentication layer on the Registry VM: a Python token service that validates OpenStack Application Credentials against Keystone and issues JWTs, fronted by Nginx as a reverse proxy.

Architecture on the Registry VM after this step:

```
External:5000 > Nginx > /auth/token > Token Service (127.0.0.1:5001)
                      > /v2/*       > Zot (127.0.0.1:5002)
```

Key commands:

```
# Generate RSA keypair for JWT signing (on host)
openssl genrsa -out auth-proxy-jwt.pem 2048
```

```
openssl rsa -in auth-proxy-jwt.pem -pubout -out auth-proxy-jwt.pub.pem

# SCP token_service.py, nginx.conf, jwt.key to Registry VM

# Inside Registry VM:
# Reconfigure Zot to listen on 127.0.0.1:5002 (free port 5000 for Nginx)
# Update /etc/zot/config.json: "address": "127.0.0.1", "port": "5002"
sudo systemctl restart zot

# Install Nginx
sudo apt-get install -y nginx

# Token Service (Python, systemd)
# KEYSTONE_URL=http://$VM_IP/identity/v3
# Listens on 127.0.0.1:5001
# Validates Application Credentials via Keystone v3 API
# Issues RS256 JWTs signed with /etc/auth-proxy/jwt.key
sudo systemctl enable --now token-service

# Nginx config:
# /auth/token > proxy_pass http://127.0.0.1:5001
# /v2/        > proxy_pass http://127.0.0.1:5002
sudo systemctl restart nginx
```

Token Service flow: 1. Client sends `GET /auth/token` with `Authorization: Basic <app_cred_id:app_cred_secret>` 2. Token service POSTs to `Keystone /auth/tokens` with the application credential 3. If Keystone returns 201, the service issues a JWT with scope-based access claims 4. Client uses the JWT as `Authorization: Bearer <token>` for `/v2/` requests

Output: Auth proxy running, `AUTH_PROXY_REALM` persisted.

Step 09 — Install K3s Cluster (Master + Workers)

File: `09_k3s_cluster_install.sh`

Installs K3s on the master and worker VMs, configured to use the private Zot registry for all image pulls.

Key commands:

```
# Create registries.yaml (mirrors ALL registries to Zot)
cat > registries.yaml <<EOF
mirrors:
  docker.io:        { endpoint: ["http://10.20.0.x:5000"] }
  registry.k8s.io:  { endpoint: ["http://10.20.0.x:5000"] }
  quay.io:          { endpoint: ["http://10.20.0.x:5000"] }
```

```
  "*":              { endpoint: ["http://10.20.0.x:5000"] }
EOF

# Generate cluster token
K3S_CLUSTER_TOKEN=$(openssl rand -hex 16)

# Create Worker VM(s) in OpenStack
openstack server create --flavor ds1G --image Ubuntu24 \
    --network k3s-cluster-net --security-group k3s-worker-sg \
    --key-name k3s-key k3s-worker-01

# Assign floating IPs to workers
openstack floating ip create public
openstack server add floating ip k3s-worker-01 <FIP>

# Install K3s Server on Master
scp k3s ubuntu@$MASTER_FIP:~/k3s
scp registries.yaml ubuntu@$MASTER_FIP:~/registries.yaml
# systemd service:
# ExecStart=/usr/local/bin/k3s server \
#   --node-ip=$MASTER_INTERNAL_IP \
#   --advertise-address=$MASTER_INTERNAL_IP \
#   --tls-san=$MASTER_FIP \
#   --token=$K3S_CLUSTER_TOKEN \
#   --disable traefik \
#   --write-kubeconfig-mode=0644
sudo systemctl enable --now k3s

# Install K3s Agent on each Worker
# systemd service:
# ExecStart=/usr/local/bin/k3s agent \
#   --server=https://$MASTER_INTERNAL_IP:6443 \
#   --token=$K3S_CLUSTER_TOKEN \
#   --node-ip=$WORKER_IP
sudo systemctl enable --now k3s-agent

# Fetch kubeconfig to host
scp ubuntu@$MASTER_FIP:/etc/rancher/k3s/k3s.yaml ./k3s-kubeconfig.yaml
sed -i "s#https://127.0.0.1:6443#https://$MASTER_FIP:6443#" ./k3s-kubeconfig.yaml
```

Output: Running K3s cluster with kubeconfig at `k3s-kubeconfig.yaml`.

Step 10 — Configure Nginx Auth Enforcement + Smoke Test

File: `10_registry_smoke_test.sh`

This is the critical step that implements **dual-mode authentication**: internal K3s pulls bypass auth, external access requires JWT.

Nginx `satisfy any` logic:

```
location /v2/ {
    satisfy any;

    # Option A: IP-based bypass (internal networks)
    allow 10.10.0.0/24;   # cluster net
    allow 10.20.0.0/24;   # services net
    deny all;

    # Option B: Bearer token validation
    auth_request /auth_validate;
    error_page 401 = /auth_error_401;
}

# Internal subrequest to token service
location = /auth_validate {
    internal;
    proxy_pass http://127.0.0.1:5001/auth/validate;
    proxy_set_header Authorization $http_authorization;
    proxy_set_header Host $http_host;  # Critical: includes port for realm URL
}

# Return proper WWW-Authenticate on 401
location = /auth_error_401 {
    internal;
    add_header WWW-Authenticate 'Bearer realm="http://$http_host/auth/token",service="zot-re
    return 401;
}
```

Token service enhancement — `/auth/validate` endpoint: - Receives the `Authorization` header from Nginx via `auth_request` - Validates the JWT signature using `/etc/zot/jwt.pub` - Checks token expiry - Returns 200 (valid) or 401 (invalid) to Nginx

Smoke tests:

```
# Test 1: External access without auth > 401
curl http://$AUTH_PROXY_FIP:5000/v2/
# Expected: 401 with WWW-Authenticate header

# Test 2: Get JWT via Application Credential
BASIC_AUTH=$(printf '%s:%s' "$APP_CRED_ID" "$APP_CRED_SECRET" | base64)
curl -H "Authorization: Basic $BASIC_AUTH" \
    "http://$AUTH_PROXY_FIP:5000/auth/token?service=zot-registry&scope=repository:sandbox/sm
```

```
# Expected: JSON with "token" field

# Test 3: Access /v2/ with Bearer token > 200
curl -H "Authorization: Bearer $JWT_TOKEN" http://$AUTH_PROXY_FIP:5000/v2/
# Expected: 200
```

Output: Auth enforcement verified — internal open, external requires JWT.

Step 11 — End-to-End Deployment Flow Demo

File: `11_enable_zot_auth.sh`

Demonstrates the complete production deployment workflow: create credentials, push an image, deploy a pod.

Key commands:

```
# 1. Create a new Application Credential for the deployment
APP_CRED_JSON=$(openstack application credential create deploy-app-$(date +%s) --unrestricte
APP_ID=$(echo "$APP_CRED_JSON" | python3 -c "import sys,json; ...")
APP_SECRET=$(echo "$APP_CRED_JSON" | python3 -c "import sys,json; ...")

# 2. Validate credential against Keystone
curl -X POST -H 'Content-Type: application/json' \
    -d '{"auth":{"identity":{"methods":["application_credential"],...}}}' \
    http://$VM_IP/identity/v3/auth/tokens
# Expected: 201

# 3. Push nginx:alpine to registry via Skopeo (from Master VM)
skopeo copy --dest-username='$APP_ID' --dest-password='$APP_SECRET' \
    --dest-tls-verify=false \
    docker://docker.io/library/nginx:alpine \
    docker://$AUTH_PROXY_FIP:5000/my-app:v1

# 4. Create Kubernetes ImagePullSecret
kubectl --kubeconfig k3s-kubeconfig.yaml create secret docker-registry registry-auth \
    --docker-server="$REGISTRY_INTERNAL_IP:5000" \
    --docker-username="$APP_ID" \
    --docker-password="$APP_SECRET"

# 5. Deploy Pod using the private registry
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: prod-app
```

```
spec:
  containers:
  - name: app
    image: $REGISTRY_INTERNAL_IP:5000/my-app:v1
  imagePullSecrets:
  - name: registry-auth
EOF

# 6. Wait for pod readiness
kubectl wait --for=condition=Ready pod/prod-app --timeout=120s
```

Output: Running pod `prod-app` pulling from the authenticated private registry.

Utility Scripts

`XX_RUN_FROM_SNAPSHOT.sh`

Full automation: reverts the DevStack VM to the `PostStack_Ready` Libvirt snapshot, resets `devstack_env` to base values, cleans generated artifacts (keys, logs, kubeconfig), and runs scripts 03-11 sequentially with logging.

`XX_reset_rerun_state.sh`

Resets `devstack_env` to only `VM_NAME`, `VM_IP`, `VM_SSH_KEY` and removes all generated files. Used to re-run from step 03 without reverting the snapshot.

`XX_revert_to_snapshot.sh`

Simple Libvirt snapshot revert:

```
sudo virsh destroy BaseDevStack
sudo virsh snapshot-revert --domain BaseDevStack --snapshotname PostStack_Ready
sudo virsh start BaseDevStack
```

`ssh-safe` / `scp-safe`

Wrappers around `ssh`/`scp` that disable strict host key checking for automation.

`source_env.sh`

Convenience script to load `devstack_env` and export SSH options for interactive use.

State Management

All scripts communicate through a single state file: **devstack_env**. Each script reads it at startup via `load_env()` and writes new variables via `persist_env()`. The state grows incrementally:

After Step	Key Variables Added
01	VM_IP
03	PROJECT_NAME, IMAGE_NAME, AUTH_PROXY_APP_CRED_ID, AUTH_PROXY_APP_CRED_SECRET
04	MASTER_NAME, MASTER_FIP, MASTER_INTERNAL_IP, MASTER_SG, KEYPAIR_NAME
05	REGISTRY_NAME, REGISTRY_INTERNAL_IP, REGISTRY_FIP
07	AUTH_PROXY_FIP, AUTH_PROXY_INTERNAL_IP
08	AUTH_PROXY_JWT_PUB_LOCAL, AUTH_PROXY_REALM
09	K3S_CLUSTER_TOKEN, WORKER_IPS, WORKER_FIP, KUBECONFIG_PATH

Key Design Decisions

1. **Air-gapped by design** — All binaries (K3s, Zot, Skopeo) are pre-downloaded on the host and SCP'd to VMs. No VM downloads from the internet.

2. **Auth enforcement at Nginx, not Zot** — Zot runs unauthenticated on localhost. Nginx enforces auth via `satisfy any` (IP allow OR Bearer token). This is essential because K3s `registries.yaml` cannot handle bearer token flows.

3. **OpenStack Application Credentials** — Instead of username/password, the system uses Keystone Application Credentials for registry auth. These are scoped, revocable, and production-realistic.

4. **Two-tenant-network architecture** — Cluster traffic (K8s) and service traffic (registry) are isolated on separate Neutron networks, both routed through a single OpenStack router.

5. **Auth Proxy merged into Registry VM** — Originally planned as a separate VM, consolidated to save resources. Nginx and Token Service run alongside Zot on the same instance.

6. **Nested virtualization** — The entire OpenStack cloud runs inside a KVM VM on the host. `cpu host-passthrough` and `LIBVIRT_TYPE=kvm` enable nested KVM for Nova instances.

Complete Command Sequence (Quick Reference)

```
# Step 01: Create base VM
./01_create_vm.sh

# Step 02: Initialize VM + transfer DevStack config
./02_init_vm.sh
# Then manually: ssh ubuntu@$VM_IP && cd ~/devstack && ./stack.sh
# Take snapshot: sudo virsh snapshot-create-as BaseDevStack PostStack_Ready

# Step 03: Bootstrap OpenStack (project, networks, credentials)
./03_openstack_bootstrap.sh

# Step 04: Create K3s master VM + floating IP + routing
./04_k3s_master_vm.sh

# Step 05: Create registry VM + Cinder volume + floating IP
./05_registry_vm.sh

# Step 06: Install Zot + push airgap images
./06_zot_install.sh

# Step 07: Alias auth proxy IPs (merged into registry VM)
./07_auth_proxy_vm.sh

# Step 08: Install Nginx + Token Service on registry VM
./08_auth_proxy_install.sh

# Step 09: Install K3s cluster (master + workers)
./09_k3s_cluster_install.sh

# Step 10: Configure auth enforcement + smoke test
./10_registry_smoke_test.sh

# Step 11: End-to-end deployment demo (push + deploy pod)
./11_enable_zot_auth.sh

# OR: Run everything from snapshot in one go
sudo ./XX_RUN_FROM_SNAPSHOT.sh
```